CYCLE D'INGÉNIEUR EN INFORMATIQUE · SPÉCIALITÉ INTELLIGENCE ARTIFICIELLE

PROJET D'INTÉGRATION EN INTELLIGENCE ARTIFICIELLE

SentinelX AI

Plateforme intelligente de maintenance prédictive
& Maintenance Copilot RAG

Cahier des charges

Spécification fonctionnelle et technique du projet — Version 1.0

Réalisé par	**[Prénom et Nom de l'étudiante]**
Encadré par	**[Nom de l'encadrant(e)]**
Établissement	**[Nom de l'école d'ingénieurs]**
Année universitaire	**2026 – 2027**
Technologies	**Angular · FastAPI · PostgreSQL · WebSocket** **scikit-learn · PyTorch · Qdrant · Docker Compose**
Durée de réalisation	**12 semaines (objectif : version V1 pour l'examen)**

Sommaire

1. Présentation générale du projet 3
- 1.1 Contexte 3
- 1.2 Problématique 3
- 1.3 Présentation de SentinelX AI et solution proposée 3
- 1.4 Objectif général et objectifs spécifiques 4

2. Étude des besoins 5
- 2.1 Utilisateurs cibles 5
- 2.2 Principaux cas d'utilisation 5
- 2.3 Besoins fonctionnels 5
- 2.4 Besoins non fonctionnels 7
- 2.5 Périmètre du projet 7

3. Organisation du projet en versions 9
- 3.1 Démarche incrémentale 9
- 3.2 Présentation des quatre versions 9
- 3.3 Contenu détaillé de la V1 (version d'examen) 9
- 3.4 Évolutions planifiées après l'examen 10

4. Fonctionnalités principales 11

5. Technologies et architecture 14
- 5.1 Stack technique 14
- 5.2 Rôle des principaux composants 14
- 5.3 Architecture générale 15
- 5.4 Communication entre les composants 16

6. Données et intelligence artificielle 17
- 6.1 Datasets utilisés 17
- 6.2 Préparation des données 17
- 6.3 Modèles de prédiction de la RUL 18
- 6.4 Détection des anomalies 19
- 6.5 Évaluation des résultats 19
- 6.6 Fonctionnement général du système RAG 20

7. Interface de l'application 21

8. Base de données 23

9. Planification 25
- 9.1 Planning de réalisation de la V1 (12 semaines) 25
- 9.2 Évolutions après l'examen 26
- 9.3 Risques et mesures prévues 27

10. Livrables et critères de validation 28

11. Conclusion 30

Références 30

1. Présentation générale du projet

1.1 Contexte

Dans le secteur industriel, la disponibilité des équipements est un enjeu majeur. Une panne non anticipée peut immobiliser une machine, interrompre la production et entraîner des coûts de réparation importants. Pour limiter ces situations, les entreprises s'appuient traditionnellement sur deux approches : la **maintenance corrective**, qui consiste à intervenir après la panne, et la **maintenance préventive**, qui repose sur des interventions planifiées à intervalles fixes, parfois trop tôt et parfois trop tard.

La **maintenance prédictive** propose une troisième voie : elle exploite les mesures des capteurs installés sur les équipements pour estimer leur état réel et anticiper le moment où une défaillance risque de se produire. Les interventions peuvent alors être planifiées au bon moment. Cette approche s'appuie naturellement sur l'intelligence artificielle, car les signes de dégradation sont souvent dissimulés dans des séries temporelles multivariées difficiles à interpréter à l'œil nu ou à l'aide de règles fixes.

1.2 Problématique

Les données de capteurs sont abondantes, mais elles restent difficiles à exploiter rapidement par les équipes de maintenance. Il ne suffit pas de disposer d'un modèle performant : il faut aussi que ses résultats soient présentés de manière claire, suivis en continu et reliés à des actions concrètes. Ce projet cherche donc à répondre à la question suivante :

> **Problématique du projet**
>
> Comment concevoir une plateforme d'intelligence artificielle capable d'anticiper la dégradation d'équipements industriels à partir de séries temporelles, de suivre leur état en temps réel et d'aider les équipes de maintenance à prioriser leurs interventions ?

1.3 Présentation de SentinelX AI et solution proposée

SentinelX AI est une plateforme web de maintenance prédictive entièrement logicielle. Elle ne nécessite aucun capteur physique : des données industrielles publiques, principalement le jeu de données **NASA C-MAPSS**, sont rejouées cycle par cycle par un simulateur afin de reproduire l'arrivée progressive des mesures comme dans une usine réelle. Ce choix rend le projet reproductible, évaluable et réalisable dans le cadre d'un projet universitaire.

À chaque nouveau cycle, la plateforme :

- estime la **durée de vie utile restante** (RUL, *Remaining Useful Life*) de l'équipement grâce à des modèles d'apprentissage automatique ;
- calcule un **score d'anomalie** indiquant si le comportement de la machine s'éloigne de son fonctionnement normal ;
- combine ces résultats en un **Health Score** compris entre 0 et 100, facile à lire ;
- déclenche des **alertes** lorsque l'état de la machine se dégrade ;

- affiche l'ensemble de ces informations en temps réel dans un **tableau de bord Angular**.

À plus long terme, la plateforme intègre un **Maintenance Copilot** basé sur la technique RAG (*Retrieval-Augmented Generation*), capable de répondre aux questions des techniciens en s'appuyant sur une base documentaire de maintenance et en citant ses sources, ainsi qu'un **Digital Twin** permettant de visualiser l'état estimé de chaque équipement.

Le développement est organisé en plusieurs versions successives (voir chapitre 3). La **version V1**, qui constitue l'objectif de réalisation pour l'examen, regroupe le cœur prédictif et le suivi en temps réel. Le copilote RAG et le Digital Twin sont prévus dans les versions ultérieures.

1.4 Objectif général et objectifs spécifiques

Objectif général

Concevoir et réaliser une application web complète, de la donnée jusqu'à l'interface, qui prépare des données industrielles, entraîne et compare plusieurs modèles de prédiction, simule le suivi d'équipements en temps réel et présente des indicateurs de santé exploitables par une équipe de maintenance.

Objectifs spécifiques

N°	Objectif	Version
OS-1	Construire une chaîne reproductible de préparation des séries temporelles (nettoyage, normalisation, fenêtrage).	MVP
OS-2	Établir un modèle de référence classique (baseline) pour la prédiction de la RUL.	MVP
OS-3	Entraîner un modèle séquentiel (LSTM ou GRU) et le comparer objectivement à la baseline.	V1
OS-4	Détecter les comportements anormaux et calculer un Health Score interprétable.	V1
OS-5	Simuler l'arrivée des données en temps réel, sans matériel physique.	MVP
OS-6	Exposer les modèles et les données via une API FastAPI documentée.	MVP
OS-7	Développer une interface Angular de suivi des machines, des alertes et de l'historique.	V1
OS-8	Conteneuriser l'application avec Docker Compose pour faciliter son installation.	V1
OS-9	Mettre en place un assistant de maintenance RAG fournissant des réponses sourcées.	V1.5
OS-10	Visualiser l'état des machines à l'aide d'un Digital Twin 3D et expliquer les prédictions.	V2

2. Étude des besoins

2.1 Utilisateurs cibles

SentinelX AI s'adresse principalement aux équipes de maintenance industrielle. Quatre profils d'utilisateurs ont été identifiés. Dans la V1, l'application fonctionne en mode démonstration, sans gestion des comptes ; les rôles seront distingués à partir de la V1.5 avec l'ajout de l'authentification.

Profil	Besoin principal	Fonctionnalités utilisées
Technicien de maintenance	Identifier rapidement les machines qui nécessitent une intervention.	Health Score, alertes, détail d'une machine, assistant IA.
Responsable maintenance	Avoir une vue globale de la flotte et prioriser les interventions.	Tableau de bord, RUL, niveaux de criticité, historique.
Ingénieur IA / données	Contrôler la qualité des modèles et suivre leurs versions.	Page de performances des modèles, suivi des expériences.
Administrateur	Gérer les utilisateurs, la base documentaire et les paramètres.	Gestion des comptes et des documents (à partir de la V1.5).

2.2 Principaux cas d'utilisation

ID	Cas d'utilisation	Version
UC-01	Lancer une simulation sur un moteur et suivre son passage d'un état sain à un état dégradé puis critique.	MVP
UC-02	Observer l'évolution de la RUL prédite à chaque nouveau cycle.	MVP
UC-03	Recevoir une alerte lorsque le Health Score passe sous un seuil défini.	V1
UC-04	Consulter le détail d'une machine : courbes des capteurs, prédictions, alertes et modèle utilisé.	V1
UC-05	Consulter l'historique des prédictions et des alertes d'une machine.	V1
UC-06	Demander à l'assistant : « Pourquoi cette machine est-elle critique ? » et obtenir une réponse sourcée.	V1.5
UC-07	Rechercher une procédure de maintenance et afficher le document source correspondant.	V1.5

2.3 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit faire. Chaque besoin est rattaché à la version dans laquelle il sera réalisé. Les besoins marqués MVP et V1 sont **obligatoires pour l'examen**.

ID	Besoin	Description	Version
BF-01	Préparation des données	Importer le dataset C-MAPSS, le nettoyer et construire la cible RUL.	MVP
BF-02	Modèle de référence	Entraîner au moins un modèle classique reproductible (baseline).	MVP
BF-03	Simulation temps réel	Rejouer les cycles d'un moteur à vitesse configurable (démarrer, mettre en pause, réinitialiser).	MVP
BF-04	Prédiction de la RUL	Calculer la RUL à chaque nouveau cycle reçu.	MVP
BF-05	Diffusion en direct	Transmettre les nouvelles valeurs au frontend par WebSocket.	MVP
BF-06	Modèle séquentiel	Entraîner un modèle LSTM ou GRU et le comparer à la baseline.	V1
BF-07	Détection d'anomalies	Calculer un score d'anomalie à partir des mesures des capteurs.	V1
BF-08	Health Score	Convertir RUL et score d'anomalie en un indicateur de 0 à 100.	V1
BF-09	Gestion des alertes	Créer des alertes « Avertissement » ou « Critique » avec horodatage et justification ; permettre leur acquittement.	V1
BF-10	Suivi des machines	Afficher la liste des machines, leur état et le détail de chacune.	V1
BF-11	Historique	Enregistrer et consulter l'historique des prédictions et des alertes.	V1
BF-12	Performances des modèles	Afficher les métriques d'évaluation et la version du modèle actif.	V1
BF-13	Base documentaire	Importer et indexer des documents de maintenance.	V1.5
BF-14	Assistant RAG	Répondre aux questions avec citation des sources, ou s'abstenir si l'information manque.	V1.5
BF-15	Authentification	Connexion des utilisateurs et rôles Administrateur / Utilisateur.	V1.5
BF-16	Suivi des expériences	Enregistrer les paramètres, métriques et versions des modèles avec MLflow.	V1.5
BF-17	Digital Twin 3D	Représenter visuellement l'équipement et son état estimé.	V2
BF-18	Explicabilité avancée	Indiquer les facteurs qui contribuent le plus à une prédiction.	V2

2.4 Besoins non fonctionnels

Les besoins non fonctionnels précisent les qualités attendues du système : performance, fiabilité, sécurité et facilité d'utilisation.

ID	Exigence	Critère attendu
BNF-01	Reproductibilité	L'application peut être lancée sur un nouveau poste à l'aide du README et de Docker Compose.
BNF-02	Réactivité	Une nouvelle mesure simulée apparaît dans l'interface en moins de 2 secondes en environnement local.
BNF-03	Documentation de l'API	Toutes les routes sont documentées automatiquement (OpenAPI / Swagger, généré par FastAPI).
BNF-04	Traçabilité	Chaque prédiction enregistre la machine, le cycle, la session et la version du modèle utilisé.
BNF-05	Robustesse	Le système affiche des messages clairs en cas de données absentes, de modèle non chargé ou de perte de connexion WebSocket.
BNF-06	Qualité du code	Code organisé en modules, typage (TypeScript côté Angular, annotations Python côté backend), tests unitaires sur les fonctions critiques.
BNF-07	Sécurité	Aucun secret dans le dépôt Git (fichier *.env*), accès à la base via un ORM, validation des entrées par Pydantic. Mots de passe hachés à partir de la V1.5.
BNF-08	Ergonomie	Les états (sain, avertissement, critique) sont signalés par une couleur accompagnée d'un texte ou d'une icône.
BNF-09	Fiabilité du RAG	À partir de la V1.5 : toute réponse documentaire doit citer sa source ; en l'absence d'information, l'assistant doit le signaler.
BNF-10	Rigueur scientifique	Aucune métrique n'est inventée ; les limites du système sont présentées clairement, et l'application est identifiée comme un prototype académique d'aide à la décision.

2.5 Périmètre du projet

Afin de garder un projet réaliste, le périmètre est défini de manière explicite.

Inclus dans le projet	Exclu du projet
Prédiction de la RUL à partir de données historiques de capteurs.	Utilisation de capteurs physiques ou de cartes électroniques.
Détection d'anomalies et calcul d'un indicateur de santé.	Commande automatique d'une machine réelle.
Simulation logicielle du temps réel.	Diagnostic certifié de sécurité industrielle.
Application web Angular, API FastAPI et base PostgreSQL.	Identification de la cause physique exacte d'une panne (les données ne le permettent pas).

Inclus dans le projet	Exclu du projet
Assistant RAG sourcé et Digital Twin (versions ultérieures).	Déploiement sur une infrastructure complexe de type Kubernetes.

Règle d'honnêteté scientifique

Une information qui n'est pas réellement prédite par un modèle (par exemple « la pièce X est défectueuse ») ne doit jamais être présentée comme un diagnostic de l'IA. L'interface parle de « facteurs contributifs » ou d'« état estimé », et non de cause certaine.

3. Organisation du projet en versions

3.1 Démarche incrémentale

SentinelX AI est un projet ambitieux qui associe apprentissage automatique, développement web, temps réel et IA générative. Pour garantir un résultat solide dans le temps imparti, le développement suit une **démarche incrémentale** : chaque version produit une application fonctionnelle et démontrable, sur laquelle la version suivante vient s'appuyer.

Ce découpage répond à un principe simple : il vaut mieux présenter un cœur de projet stable, évalué et bien intégré qu'un grand nombre de fonctionnalités inachevées. L'**objectif principal à court terme est la V1**, qui sera réalisée et présentée lors de l'examen universitaire. Les versions V1.5 et V2 constituent des évolutions planifiées, développées progressivement après l'examen.

3.2 Présentation des quatre versions

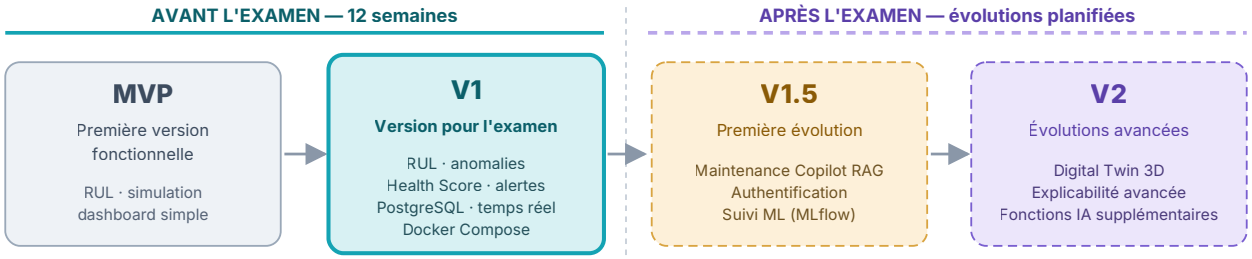


Figure 1 — Feuille de route de SentinelX AI

Version	Contenu	Statut
MVP	Première version fonctionnelle : préparation de C-MAPSS (FD001), modèle de référence pour la RUL, API FastAPI, simulateur de données et dashboard Angular simple affichant la RUL en direct.	**Obligatoire** — jalon intermédiaire (fin de semaine 6)
V1	Application Angular connectée à FastAPI et PostgreSQL, avec modèle séquentiel LSTM/GRU, détection d'anomalies, Health Score, alertes, historique et suivi des machines en temps réel. Conteneurisation Docker Compose et tests.	**Obligatoire** — version présentée à l'examen (fin de semaine 12)
V1.5	Intégration du Maintenance Copilot RAG (Qdrant + LLM) avec citations et abstention, authentification avec rôles, suivi des expériences avec MLflow.	Évolution planifiée après l'examen
V2	Digital Twin 3D, explicabilité avancée des prédictions, amélioration de la recherche documentaire et fonctionnalités IA supplémentaires.	Évolution planifiée après l'examen

3.3 Contenu détaillé de la V1 (version d'examen)

La V1 représente l'objectif principal de réalisation. Elle doit permettre de démontrer une chaîne complète, depuis les données brutes jusqu'à l'affichage en temps réel dans l'interface. Elle comprend les éléments suivants :

Domaine	Éléments inclus dans la V1
Données	Dataset C-MAPSS FD001 préparé, analyse exploratoire, séparation entraînement / validation / test par moteur.
Intelligence artificielle	Baseline classique (scikit-learn), modèle LSTM ou GRU (PyTorch), détection d'anomalies, Health Score, rapport de comparaison des modèles.
Backend	API FastAPI documentée, simulateur intégré, service d'inférence, gestion des alertes, diffusion WebSocket.
Base de données	PostgreSQL : machines, sessions de simulation, mesures, prédictions et alertes.
Frontend	Application Angular : dashboard, liste et détail des machines, simulateur, alertes, performances des modèles.
Qualité et déploiement	Tests unitaires et d'intégration, Docker Compose, README d'installation.

> **À retenir**
>
> Le Maintenance Copilot RAG et le Digital Twin font partie de la vision globale de SentinelX AI, mais **ils ne sont pas nécessaires pour valider la V1**. L'architecture de la V1 est toutefois conçue pour pouvoir les accueillir sans remise en cause de l'existant.

3.4 Évolutions planifiées après l'examen

La V1.5 ajoute la couche d'IA générative. Le Maintenance Copilot permettra d'interroger une base documentaire de maintenance (procédures, fiches techniques) et de relier les prédictions de la plateforme à des recommandations sourcées. L'authentification introduira des comptes et des rôles, et MLflow permettra de conserver un historique rigoureux des expériences et des versions de modèles.

La V2 regroupe les fonctionnalités les plus avancées : un Digital Twin 3D synchronisé avec le flux de données, des méthodes d'explicabilité permettant d'indiquer les capteurs qui influencent le plus une prédiction, une recherche documentaire améliorée (recherche hybride, reclassement des résultats) et, si le temps le permet, des modules complémentaires sur d'autres jeux de données.

4. Fonctionnalités principales

Ce chapitre décrit le rôle et le fonctionnement de chaque fonctionnalité de SentinelX AI. La version de réalisation est indiquée pour chacune d'elles.

4.1 Tableau de bord de surveillance des machines V1

Rôle. Le tableau de bord offre une vue d'ensemble de la flotte de machines simulées. Il permet de repérer en un coup d'œil les équipements qui nécessitent une attention particulière.

Fonctionnement. La page affiche des indicateurs globaux (nombre de machines suivies, Health Score moyen, nombre d'alertes actives, RUL moyenne), ainsi que la liste des machines avec leur état coloré. Les valeurs sont mises à jour automatiquement grâce à la connexion WebSocket, sans rechargement de la page. Une version simplifiée, limitée à l'affichage de la RUL, est disponible dès le MVP.

4.2 Simulation des données industrielles en temps réel MVP

Rôle. Le simulateur remplace les capteurs physiques. Il reproduit l'arrivée progressive des mesures d'un moteur, comme si celui-ci fonctionnait réellement.

Fonctionnement. L'utilisateur choisit un moteur du dataset C-MAPSS et une vitesse de rejeu (par exemple 1 cycle par seconde). Le simulateur, intégré au backend FastAPI, lit les cycles un par un et les transmet au service de prédiction. L'utilisateur peut démarrer, mettre en pause ou réinitialiser la simulation. Il est ainsi possible d'observer en quelques minutes le passage d'un moteur d'un état sain à un état critique.

4.3 Prédiction des pannes et de la durée de vie restante (RUL) MVP V1

Rôle. Cette fonctionnalité constitue le cœur scientifique du projet. Elle estime le nombre de cycles de fonctionnement restants avant la défaillance du moteur.

Fonctionnement. À chaque nouveau cycle, le backend constitue une fenêtre contenant les dernières mesures du moteur et la transmet au modèle entraîné, qui retourne une valeur de RUL. Le MVP utilise un modèle de référence classique ; la V1 introduit un modèle séquentiel (LSTM ou GRU), mieux adapté aux séries temporelles. La prédiction est enregistrée en base avec la version du modèle utilisée, puis envoyée à l'interface.

4.4 Détection des anomalies V1

Rôle. La détection d'anomalies complète la RUL en signalant les comportements inhabituels, même lorsque la durée de vie estimée reste élevée.

Fonctionnement. Un modèle apprend le comportement normal du moteur à partir des premiers cycles, considérés comme sains. Pour chaque nouvelle mesure, il calcule un score indiquant à quel point elle s'éloigne de ce comportement de référence. Plus le score est élevé, plus le comportement est jugé anormal.

4.5 Calcul du Health Score V1

Rôle. Le Health Score résume l'état de la machine en un seul indicateur compris entre 0 et 100, compréhensible par tous les utilisateurs.

Fonctionnement. Il combine la RUL normalisée et le score d'anomalie selon une formule pondérée, par exemple :

> **Formule de départ (paramétrable)**
> Health Score = 100 × [0,6 × RUL normalisée + 0,4 × (1 − score d'anomalie normalisé)]

Il est important de préciser que le Health Score n'est **pas une valeur apprise** : c'est un indicateur construit à partir des sorties des modèles. Les poids et les seuils sont des choix de conception, documentés et ajustés à l'aide des données de validation. Trois états sont définis :

État	Health Score (valeurs initiales)	Comportement de l'interface
● Sain	≥ 70	Affichage en vert, aucune alerte.
● Avertissement	entre 40 et 69	Affichage en orange, alerte de niveau « Avertissement », surveillance de la tendance.
● Critique	< 40	Affichage en rouge, alerte « Critique », inspection recommandée.

4.6 Gestion des alertes V1

Rôle. Les alertes informent l'utilisateur dès qu'une machine change d'état, afin qu'il puisse réagir rapidement.

Fonctionnement. Lorsque le Health Score franchit un seuil, le backend crée une alerte contenant le niveau (Avertissement ou Critique), la machine concernée, le cycle, l'horodatage et une courte justification (par exemple « RUL estimée inférieure à 30 cycles et score d'anomalie en hausse »). L'alerte est enregistrée dans PostgreSQL et transmise immédiatement à l'interface. L'utilisateur peut consulter la liste des alertes, les filtrer et les marquer comme traitées (acquittement).

4.7 Consultation de l'historique des machines V1

Rôle. L'historique permet d'analyser l'évolution d'une machine dans le temps et de justifier une décision de maintenance.

Fonctionnement. Toutes les prédictions et alertes sont enregistrées dans la base de données. Depuis la page de détail d'une machine, l'utilisateur visualise sous forme de graphiques l'évolution de la RUL, du Health Score et du score d'anomalie, ainsi que la chronologie des alertes, pour chaque session de simulation.

4.8 Assistant intelligent de maintenance basé sur RAG V1.5

Rôle. Le Maintenance Copilot aide le technicien à comprendre la situation d'une machine et à trouver la procédure adaptée, en s'appuyant sur une documentation fiable plutôt que sur les connaissances générales d'un modèle de langage.

Fonctionnement. L'utilisateur pose une question en langage naturel, par exemple « Pourquoi cette machine est-elle critique et que dois-je vérifier ? ». Le système récupère l'état actuel de la machine via l'API, recherche dans la base documentaire les passages les plus pertinents, puis demande au LLM de formuler une réponse fondée uniquement sur ces éléments. La réponse est accompagnée de ses sources (document, page ou section). Si la documentation ne contient pas l'information, l'assistant l'indique clairement au lieu d'inventer une réponse.

4.9 Visualisation de l'état des machines avec un Digital Twin V2

Rôle. Le Digital Twin offre une représentation visuelle de l'équipement qui reflète son état estimé, afin de rendre le suivi plus intuitif.

Fonctionnement. Il s'agit d'un jumeau numérique *orienté données* : une représentation 3D simplifiée du moteur, synchronisée avec le flux simulé, dont l'apparence change selon l'état (vert, orange, rouge, ou gris en l'absence de données). Ce n'est pas une simulation physique haute-fidélité. Le modèle 3D utilisé sera libre de droits ou réalisé spécifiquement pour le projet.

5. Technologies et architecture

5.1 Stack technique

Couche	Technologies	Utilisation dans le projet	Version
Frontend	Angular, TypeScript, Angular Material, bibliothèque de graphiques (ex. ng2-charts)	Interface web, pages, graphiques en temps réel.	MVP
Backend	Python, FastAPI, Pydantic, SQLAlchemy, Uvicorn	API REST, WebSocket, simulateur, logique métier.	MVP
Base de données	PostgreSQL	Stockage des machines, sessions, prédictions et alertes.	V1
Intelligence artificielle	Python, pandas, NumPy, scikit-learn, PyTorch	Préparation des données, modèles RUL et détection d'anomalies.	MVP
Temps réel	WebSocket (FastAPI côté serveur, RxJS côté Angular)	Envoi instantané des nouvelles valeurs à l'interface.	MVP
Déploiement	Docker, Docker Compose	Lancement de l'ensemble des services en une commande.	V1
RAG	Qdrant, modèle d'embeddings, LLM	Base vectorielle et génération des réponses de l'assistant.	V1.5
Suivi des expériences	MLflow	Historique des paramètres, métriques et versions de modèles.	V1.5
Outils	Git / GitHub, Jupyter, pytest	Versionnement du code, exploration des données, tests.	—

5.2 Rôle des principaux composants

Angular — l'interface utilisateur

Angular est utilisé pour construire une application web monopage (SPA) structurée et maintenable. L'application est organisée en **composants** (une page ou un élément d'interface), en **services** (communication avec l'API REST et le WebSocket) et en **routes** (navigation entre les pages). TypeScript apporte un typage strict qui limite les erreurs, notamment pour les objets échangés avec le backend. La bibliothèque RxJS, intégrée à Angular, permet de traiter le flux de données reçu en continu par WebSocket et de mettre à jour automatiquement les graphiques.

FastAPI — le cœur applicatif

FastAPI a été choisi car il est écrit en Python, le même langage que les modèles d'IA, ce qui simplifie leur intégration. Il permet de définir rapidement des routes REST et des connexions WebSocket, valide automatiquement les données grâce à Pydantic et génère une documentation interactive (Swag-

ger). Le backend regroupe l'API, le simulateur, le service d'inférence qui charge les modèles entraînés, le calcul du Health Score et la gestion des alertes.

PostgreSQL — la persistance des données

PostgreSQL est une base de données relationnelle robuste et open source. Elle conserve les informations nécessaires au suivi et à la traçabilité : machines, sessions de simulation, mesures, prédictions et alertes. L'accès à la base se fait via l'ORM SQLAlchemy, ce qui évite d'écrire des requêtes SQL à la main et protège contre les injections SQL.

Les modèles d'intelligence artificielle

Les modèles sont entraînés hors ligne dans des scripts Python (scikit-learn pour la baseline et la détection d'anomalies, PyTorch pour le modèle séquentiel). Une fois évalués, ils sont sauvegardés sous forme de fichiers (modèle, normaliseur, configuration) que le backend charge au démarrage. Le backend n'entraîne donc pas les modèles en direct : il les utilise uniquement pour produire des prédictions.

5.3 Architecture générale

L'application suit une architecture en trois niveaux classique (présentation, logique applicative, données), complétée par les modèles d'IA. Tous les services sont lancés ensemble grâce à Docker Compose.

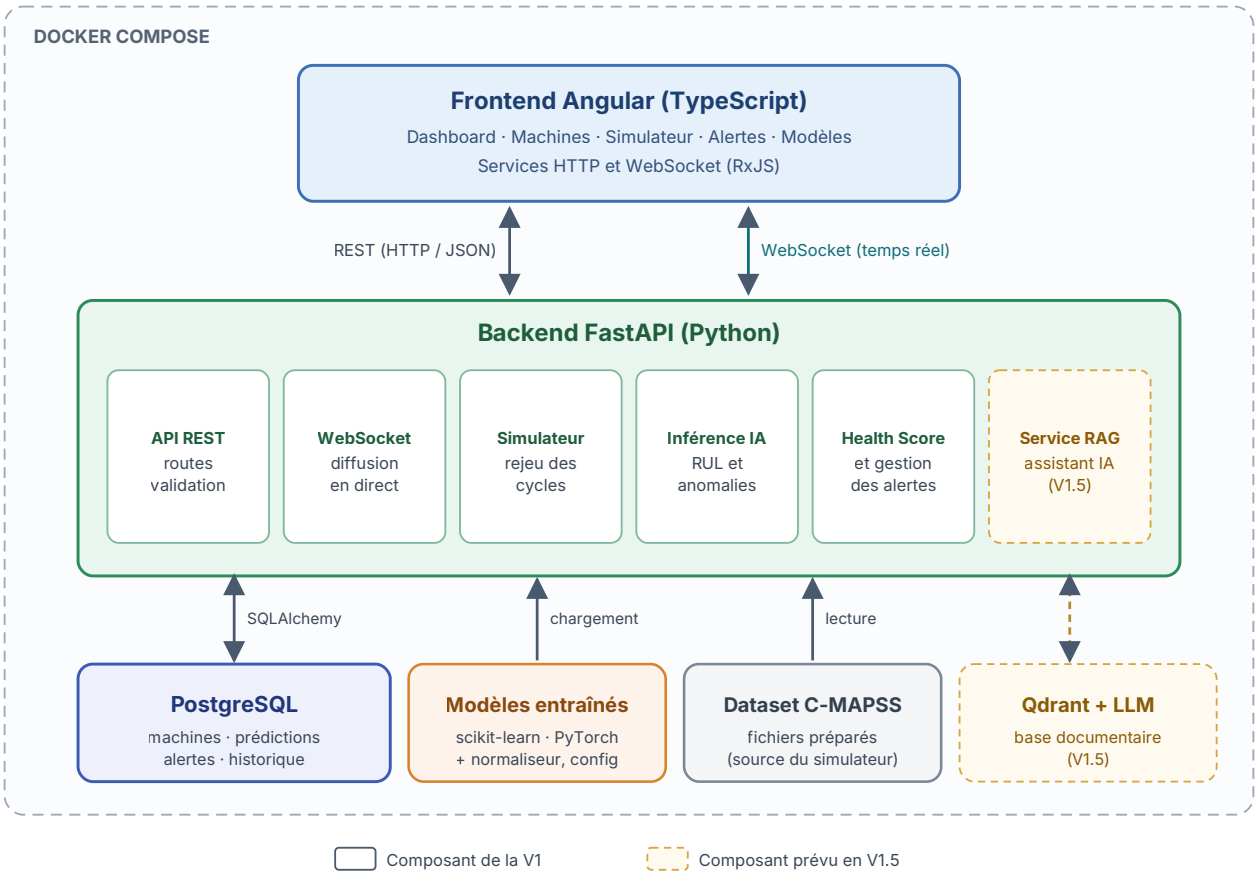


Figure 2 — Architecture générale de SentinelX AI

5.4 Communication entre les composants

Le frontend et le backend communiquent selon deux modes complémentaires :

- **REST (HTTP/JSON)** pour les actions ponctuelles : obtenir la liste des machines, consulter l'historique, démarrer une simulation, acquitter une alerte.
- **WebSocket** pour le flux continu : à chaque cycle simulé, le backend envoie immédiatement la nouvelle mesure, la RUL, le score d'anomalie, le Health Score et l'éventuelle alerte. Le frontend gère les reconnexions et affiche l'heure de la dernière donnée reçue.

Le déroulement d'un cycle de simulation est le suivant :

1. L'utilisateur démarre une simulation depuis la page « Simulateur » (requête REST).
2. Le simulateur lit le cycle suivant du moteur choisi dans le dataset préparé.
3. Le service d'inférence construit la fenêtre temporelle et calcule la RUL et le score d'anomalie.
4. Le Health Score est calculé ; une alerte est créée si un seuil est franchi.
5. La mesure, la prédiction et l'alerte éventuelle sont enregistrées dans PostgreSQL.
6. Le résultat est envoyé par WebSocket au frontend, qui met à jour les graphiques et les indicateurs.

Principales routes de l'API

Méthode	Route	Rôle	Version
GET	/health	État de l'API et des modèles chargés	MVP
GET	/machines, /machines/{id}	Liste des machines et état courant	V1
POST	/simulations	Créer une session de simulation	MVP
POST	/simulations/{id}/start · pause · reset	Contrôler la simulation	MVP
GET	/machines/{id}/predictions	Historique des prédictions	V1
GET · PATCH	/alerts, /alerts/{id}	Consulter et acquitter les alertes	V1
GET	/models/metrics	Métriques et version du modèle actif	V1
WS	/ws/machines/{id}	Flux temps réel d'une machine	MVP
POST	/auth/login	Connexion	V1.5
POST · GET	/documents	Importer et lister les documents	V1.5
POST	/copilot/query	Poser une question à l'assistant	V1.5

6. Données et intelligence artificielle

6.1 Datasets utilisés

Dataset principal : NASA C-MAPSS

Le jeu de données **C-MAPSS** (*Commercial Modular Aero-Propulsion System Simulation*), publié par le centre de pronostic de la NASA, est une référence pour la prédiction de la RUL. Il contient des séries temporelles simulées de moteurs d'avion à turboréacteur. Chaque moteur est décrit cycle par cycle par 3 paramètres de conditions opérationnelles et 21 mesures de capteurs (températures, pressions, vitesses de rotation, etc.).

Dans les données d'entraînement, chaque moteur est suivi jusqu'à sa défaillance. Dans les données de test, la série s'arrête avant la panne et la RUL réelle est fournie séparément, ce qui permet une évaluation objective. Le dataset est divisé en quatre sous-ensembles de difficulté croissante :

Sous-ensemble	Conditions opérationnelles	Modes de panne	Utilisation dans le projet
FD001	1	1	Sous-ensemble principal du MVP et de la V1.
FD002	6	1	Test de robustesse à des conditions variées (si le temps le permet).
FD003	1	2	Diversification des types de dégradation (extension).
FD004	6	2	Cas le plus difficile, envisagé dans les versions ultérieures.

Datasets complémentaires (versions ultérieures)

Deux jeux de données pourront enrichir le projet après la V1 : **AI4I 2020** (UCI), qui contient 10 000 observations synthétiques de maintenance prédictive et se prête à un module de classification des défaillances, et **MIMII**, qui regroupe des enregistrements sonores normaux et anormaux de machines industrielles. Ces données ne sont pas synchronisées avec C-MAPSS : elles seraient traitées dans des modules séparés, sans prétendre à une fusion multimodale.

6.2 Préparation des données

La préparation des données suit les étapes illustrées ci-dessous. L'exploration initiale est réalisée dans des notebooks Jupyter, puis la logique retenue est reprise dans des scripts Python reproductibles.



Figure 3 — Étapes de préparation des données

- **Exploration et nettoyage :** analyse des distributions, des corrélations et de l'évolution de chaque capteur au cours de la vie du moteur. Les capteurs constants ou peu informatifs sont écartés.
- **Construction de la cible RUL :** pour chaque cycle, la RUL correspond au nombre de cycles restants avant la fin de la série du moteur. Une version plafonnée sera testée, car il est peu utile de distinguer précisément deux moteurs encore très éloignés de la panne ; la valeur du plafond sera choisie par validation.
- **Découpage des données :** la séparation entre entraînement, validation et test se fait **par moteur** et non par ligne. Un découpage aléatoire mélangerait les cycles d'un même moteur entre les ensembles et surestimerait artificiellement les performances (fuite de données).
- **Normalisation et fenêtrage :** les mesures sont mises à la même échelle à partir des seules données d'entraînement. Pour le modèle séquentiel, les données sont découpées en fenêtres glissantes de plusieurs cycles consécutifs.

6.3 Modèles de prédiction de la RUL

La démarche consiste à toujours établir un modèle simple avant un modèle plus complexe, afin de mesurer objectivement l'apport de ce dernier.

Modèle	Bibliothèque	Principe et intérêt	Version
Régression linéaire (Ridge)	scikit-learn	Modèle très simple et interprétable, utilisé comme premier point de comparaison.	MVP
Random Forest	scikit-learn	Ensemble d'arbres de décision capable de capturer des relations non linéaires ; baseline principale à partir de caractéristiques calculées sur une fenêtre (moyennes, tendances).	MVP
LSTM ou GRU	PyTorch	Réseaux de neurones récurrents conçus pour les séquences : ils reçoivent directement une fenêtre de plusieurs cycles et apprennent l'évolution temporelle de la dégradation.	V1

Pour le modèle séquentiel, quelques hyperparamètres seront explorés de manière raisonnable : longueur de la fenêtre (par exemple 20, 30 ou 50 cycles), taille de la couche cachée, nombre de couches et taux de *dropout*. L'entraînement utilisera l'arrêt anticipé (*early stopping*) sur l'ensemble de validation afin de limiter le surapprentissage.

> **Remarque**
>
> Il est possible que le modèle séquentiel n'apporte pas de gain significatif par rapport à la baseline. Ce résultat serait tout aussi valable scientifiquement : il sera présenté et analysé, et le modèle le plus adapté sera retenu pour l'application.

6.4 Détection des anomalies

Le dataset C-MAPSS ne contient pas d'étiquettes d'anomalies explicites. L'approche retenue consiste donc à apprendre le **comportement normal** du moteur, puis à mesurer l'écart des nouvelles observations par rapport à cette référence :

- **Approche retenue pour la V1 :** un modèle non supervisé simple, tel que l'*Isolation Forest* de scikit-learn, entraîné sur les premiers cycles des moteurs, considérés comme sains. Il fournit un score d'anomalie pour chaque nouvelle mesure.
- **Amélioration envisagée :** un auto-encodeur (PyTorch) entraîné à reconstruire des données saines ; l'erreur de reconstruction servirait alors de score d'anomalie.

Pour évaluer cette détection, un état « dégradé » sera défini à partir de la RUL réelle (par exemple, RUL inférieure à un seuil donné). Il sera alors possible de mesurer la capacité du score à repérer ces situations.

6.5 Évaluation des résultats

Les modèles seront évalués sur un ensemble de test jamais utilisé pendant l'entraînement ni le réglage des paramètres. Les métriques suivantes sont prévues :

Tâche	Métriques	Signification
Prédiction de la RUL	MAE, RMSE, R^2 (score NASA en complément)	Erreur moyenne en nombre de cycles ; le RMSE pénalise davantage les grosses erreurs. Le score NASA pénalise plus fortement les prédictions trop optimistes.
Détection d'anomalies	Précision, rappel, F1-score	Capacité à détecter les états dégradés sans générer trop de fausses alertes.
Temps réel	Latence moyenne	Temps entre la réception d'un cycle et son affichage dans l'interface.
Assistant RAG V1.5	Pertinence des passages, exactitude des citations, taux d'abstention correcte	Évaluées sur un jeu de 30 à 50 questions de référence.

Le protocole d'évaluation prévoit de comparer la baseline et le modèle séquentiel sur le même découpage, de répéter l'entraînement avec plusieurs graines aléatoires lorsque le temps le permet, d'analyser les erreurs selon la phase de vie du moteur (début, milieu, proximité de la panne) et de tracer les prédictions face aux valeurs réelles pour plusieurs moteurs. Les résultats seront rapportés tels qu'ils seront obtenus, sans valeur fixée à l'avance.

6.6 Fonctionnement général du système RAG V1.5

Un modèle de langage (LLM) seul peut produire des réponses plausibles mais inexactes, ce qui est inacceptable dans un contexte de maintenance. La technique **RAG** (*Retrieval-Augmented Generation*) limite ce risque : avant de générer une réponse, le système recherche les passages pertinents dans une base documentaire contrôlée et demande au LLM de s'appuyer uniquement sur ces passages.

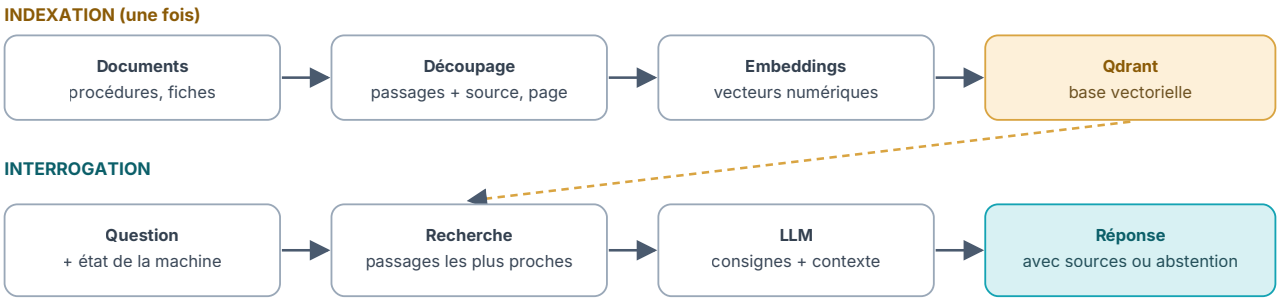


Figure 4 — Fonctionnement du Maintenance Copilot (RAG)

Le système fonctionne en deux temps. Lors de l'**indexation**, les documents autorisés (procédures rédigées pour le projet, documentation libre de droits) sont découpés en passages, chacun associé à sa source et à sa page. Ces passages sont transformés en vecteurs (*embeddings*) puis stockés dans Qdrant. Lors de l'**interrogation**, la question est elle aussi transformée en vecteur ; Qdrant retrouve les passages les plus proches, qui sont transmis au LLM avec l'état actuel de la machine.

Le LLM suit des règles strictes : répondre uniquement à partir du contexte fourni, citer systématiquement ses sources, indiquer « Information insuffisante dans la base documentaire » lorsque c'est le cas, traiter le contenu des documents comme des données et non comme des instructions, et ne formuler que des recommandations informatives, sans aucune action automatique. L'architecture restera indépendante du fournisseur de LLM, afin de pouvoir utiliser un modèle local si nécessaire.

7. Interface de l'application

L'interface est développée avec Angular. Elle privilégie la lisibilité : chaque état est signalé par une couleur accompagnée d'un libellé, chaque alerte explique sa cause en une phrase, et la version du modèle ainsi que l'heure de la dernière mise à jour sont toujours visibles. Le nombre de points affichés sur les graphiques en direct est limité pour garder un affichage fluide.

7.1 Principales pages

Page	Contenu principal	Version
Dashboard	Indicateurs globaux (machines suivies, Health Score moyen, alertes actives, RUL moyenne), état de chaque machine, dernières alertes.	V1
Liste des machines	Tableau des machines avec Health Score, RUL, état et dernière mise à jour ; tri et filtrage par état.	V1
Détail d'une machine	Graphiques des principaux capteurs, évolution de la RUL, du score d'anomalie et du Health Score, chronologie des alertes, modèle utilisé, historique des sessions.	V1
Simulateur	Choix du moteur, réglage de la vitesse, boutons démarrer / pause / réinitialiser, suivi du cycle en cours.	MVP
Alertes	Liste des alertes avec niveau, machine, cycle, date et justification ; filtres et acquittement.	V1
Performances des modèles	Comparaison baseline / modèle séquentiel, métriques de test, version du modèle actif.	V1
Assistant IA	Zone de discussion, rappel de l'état de la machine sélectionnée, sources affichées sous chaque réponse.	V1.5
Documents	Import de documents, statut d'indexation, suppression (administrateur).	V1.5
Connexion	Authentification par e-mail et mot de passe.	V1.5
Digital Twin	Vue 3D de l'équipement colorée selon son état estimé.	V2

7.2 Maquette indicative du dashboard

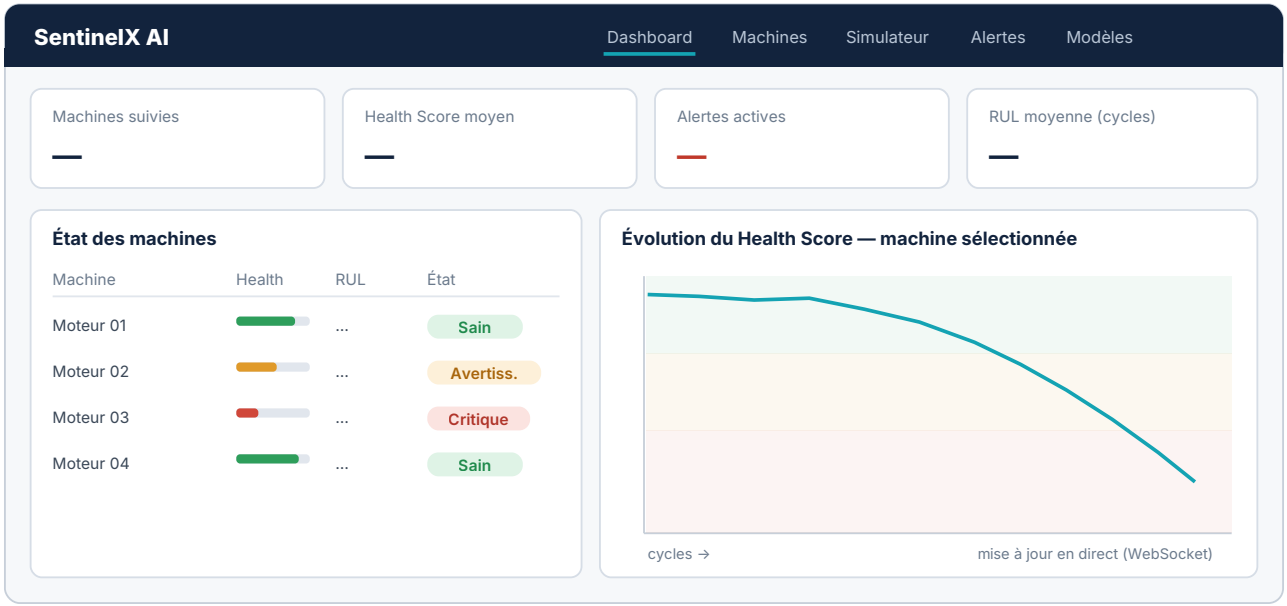


Figure 5 — Maquette indicative du dashboard (sans valeurs réelles)

7.3 Organisation de l'application Angular

L'application est découpée en **modules fonctionnels** correspondant aux pages (dashboard, machines, simulateur, alertes, modèles), complétés par un module partagé contenant les composants réutilisables (carte d'indicateur, badge d'état, graphique temps réel). Les échanges avec le backend sont centralisés dans des services Angular : un service par ressource REST et un service dédié au WebSocket, qui expose le flux de données sous forme d'*Observable* RxJS. Les modèles de données TypeScript (Machine, Prédiction, Alerte) reprennent la structure des schémas Pydantic du backend, afin d'assurer la cohérence des échanges.

8. Base de données

La base de données PostgreSQL assure la persistance et la traçabilité des informations produites par la plateforme. Son schéma est volontairement simple : il couvre les besoins de la V1 et peut être étendu pour les versions suivantes sans modification des tables existantes.

8.1 Tables principales

Table	Champs principaux	Rôle	Version
machines	id, nom, sous_ensemble, id_moteur_source, description	Catalogue des machines simulées (moteurs C-MAPSS).	MVP
simulation_sessions	id, machine_id, vitesse, cycle_depart, statut, date_debut, date_fin	Suivi des sessions de simulation et de leur état.	V1
sensor_readings	id, session_id, cycle, mesures (JSON), horodatage	Mesures reçues à chaque cycle, utilisées pour les graphiques.	V1
predictions	id, session_id, cycle, rul, score_anomalie, health_score, niveau, version_modele, horodatage	Résultats des modèles ; garantit la traçabilité de chaque prédiction.	V1
alerts	id, prediction_id, niveau, justification, acquittee, horodatage	Alertes générées et leur statut de traitement.	V1
model_versions	id, nom, type, version, metriques (JSON), actif, date_creation	Liste des modèles disponibles et de leurs performances.	V1
users	id, email, mot_de_passe_hache, role, date_creation	Comptes utilisateurs et rôles.	V1.5
documents	id, nom, version, type_source, empreinte, date_indexation	Références des documents de la base RAG.	V1.5
chat_sessions	id, user_id, machine_id, date_creation	Conversations avec l'assistant.	V1.5
chat_messages	id, chat_session_id, role, contenu, citations (JSON), horodatage	Questions, réponses et sources citées.	V1.5

8.2 Relations entre les données

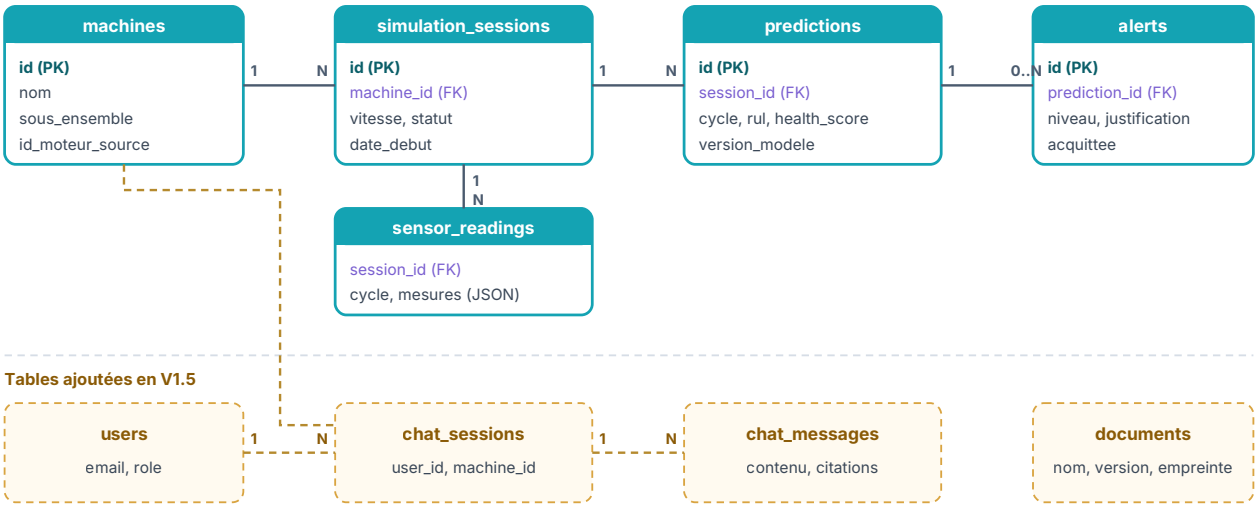


Figure 6 — Schéma simplifié des relations entre les tables

Les relations essentielles sont les suivantes :

- une **machine** peut faire l'objet de plusieurs **sessions de simulation** ;
- une **session** produit une mesure et une **prédiction** à chaque cycle ;
- une **prédiction** peut donner lieu à une **alerte** lorsque le Health Score franchit un seuil ;
- chaque prédiction conserve la **version du modèle** utilisée, ce qui garantit la traçabilité des résultats ;
- à partir de la V1.5, un **utilisateur** peut ouvrir plusieurs **conversations** avec l'assistant, chacune éventuellement liée à une machine.

Les vecteurs des documents ne sont pas stockés dans PostgreSQL mais dans Qdrant ; la table *documents* conserve uniquement leurs références. Le schéma est créé et versionné à l'aide de migrations (Alembic), et un script d'initialisation insère les machines de démonstration.

9. Planification

9.1 Planning de réalisation de la V1 (12 semaines)

Le planning couvre les douze semaines précédant l'examen. Il est construit de façon à valider d'abord le cœur scientifique (données et modèles) avant de développer l'interface. Deux jalons structurent cette période : le **MVP à la fin de la semaine 6** et la **V1 à la fin de la semaine 12**.

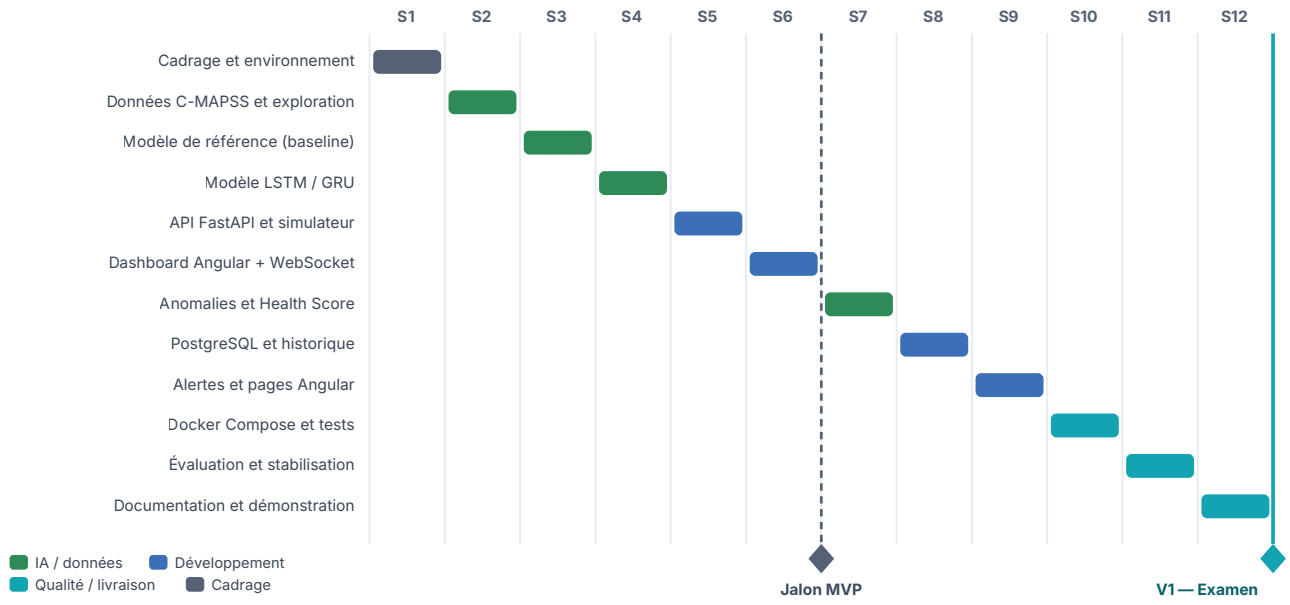


Figure 7 — Diagramme de Gantt de la réalisation de la V1

Sem.	Étape	Tâches principales	Résultat attendu
S1	Cadrage et environnement	Validation du cahier des charges, création du dépôt GitHub, mise en place des environnements Python et Angular, choix de FD001.	Dépôt structuré, README initial, backlog des tâches.
S2	Données et exploration	Téléchargement de C-MAPSS, dictionnaire des colonnes, visualisations, découpage par moteur, script de préparation.	Notebook d'exploration et données préparées.
S3	Baseline	Construction de la cible RUL, caractéristiques par fenêtre, Ridge et Random Forest, calcul des métriques.	Baseline entraînée et métriques enregistrées.
S4	Modèle séquentiel	Préparation des séquences, entraînement LSTM/GRU, arrêt anticipé, comparaison avec la baseline.	Modèle sauvegardé et rapport comparatif.
S5	API et simulateur	Projet FastAPI, chargement du modèle, route de prédiction, simulateur avec démarrage / pause / réinitialisation.	API testable via Swagger.

Sem.	Étape	Tâches principales	Résultat attendu
S6	Dashboard Angular	Création du projet Angular, page simulateur, graphique RUL en direct, connexion WebSocket.	**MVP démontrable.**
S7	Anomalies et Health Score	Modèle de détection d'anomalies, formule du Health Score, choix des seuils, tests unitaires.	Module d'évaluation de l'état testé.
S8	PostgreSQL	Schéma de la base, migrations, enregistrement des sessions, mesures et prédictions, routes d'historique.	Données persistées et consultables.
S9	Alertes et interface	Génération et acquittement des alertes, pages Dashboard, Machines, Détail, Alertes et Performances des modèles.	Interface V1 complète.
S10	Conteneurisation et tests	Dockerfiles et Docker Compose (frontend, backend, PostgreSQL), tests d'intégration du chemin complet.	Application lancée en une commande.
S11	Évaluation et stabilisation	Évaluation finale sur le jeu de test, analyse des erreurs, mesure de la latence, corrections.	Version candidate stable et résultats finaux.
S12	Préparation de l'examen	Rapport, captures d'écran, scénario de démonstration, vidéo de secours, support de présentation.	**V1 livrée et présentée.**

> **Règle de priorisation en cas de retard**
>
> Si un retard de plus d'une semaine est constaté à la fin de la semaine 8, la page « Performances des modèles » sera simplifiée et l'auto-encodeur ne sera pas étudié. En revanche, l'évaluation rigoureuse des modèles, la démonstration de bout en bout et la gestion des alertes ne seront jamais sacrifiées.

9.2 Évolutions après l'examen

Une fois la V1 validée, le projet se poursuivra par étapes. Les durées indiquées sont des estimations, à ajuster selon la disponibilité.

Version	Durée estimée	Étapes principales	Résultat attendu
V1.5	4 à 5 semaines	1. Constitution du corpus documentaire et indexation dans Qdrant. 2. Service RAG : recherche, consignes du LLM, citations, abstention. 3. Page Assistant IA et page Documents dans Angular. 4. Authentification et rôles. 5. Suivi des expériences avec MLflow. 6. Benchmark de 30 à 50 questions.	Assistant de maintenance sourcé, intégré et évalué.

Version	Durée estimée	Étapes principales	Résultat attendu
V2	4 à 6 semaines	1. Digital Twin 3D synchronisé avec le flux. 2. Explicabilité (importance des capteurs, facteurs contributifs). 3. Recherche hybride et reclassement dans le RAG. 4. Extension aux sous-ensembles FD002 à FD004 ou à AI4I.	Version enrichie de la plateforme.

9.3 Risques et mesures prévues

Risque	Impact	Mesure prévue
Périmètre trop large	Élevé	Respect strict du découpage en versions ; aucune fonctionnalité V1.5 ou V2 avant la fin de la V1.
Fuite de données	Élevé	Découpage par moteur et normalisation calculée uniquement sur les données d'entraînement.
Modèle LSTM/GRU moins performant que la baseline	Moyen	Résultat présenté et analysé honnêtement ; le meilleur modèle est utilisé dans l'application.
Difficultés d'intégration Angular / FastAPI	Moyen	Définition des schémas d'échange dès la semaine 5 et intégration progressive à partir du MVP.
Problèmes de conteneurisation en fin de projet	Élevé	Dockerfiles créés dès les premières semaines de développement, puis complétés progressivement.
Démonstration perturbée le jour de l'examen	Élevé	Fonctionnement entièrement local, données de démonstration préchargées et vidéo de secours.
LLM indisponible ou payant (V1.5)	Moyen	Architecture indépendante du fournisseur ; possibilité d'utiliser un modèle local.

10. Livrables et critères de validation

10.1 Livrables de la V1 (examen)

Livrable	Description
Application web fonctionnelle	Frontend Angular connecté au backend, avec dashboard, machines, simulateur, alertes et performances des modèles.
API FastAPI	Routes REST et WebSocket documentées automatiquement (Swagger).
Modèles IA entraînés et évalués	Baseline, modèle séquentiel et détection d'anomalies, avec scripts d'entraînement reproductibles et rapport d'évaluation.
Base de données	Schéma PostgreSQL, migrations et script d'initialisation des données de démonstration.
Code source	Dépôt GitHub organisé (frontend, backend, ml, données d'exemple, scripts), sans secrets ni jeux de données volumineux.
Déploiement	Fichier Docker Compose permettant de lancer l'ensemble de l'application.
Tests	Tests unitaires (cible RUL, fenêtrage, Health Score, seuils) et tests d'intégration (simulateur → modèle → base → WebSocket).
Documentation	README d'installation, présent cahier des charges, rapport de projet présentant la démarche, les résultats et les limites.
Démonstration finale	Scénario de démonstration, support de présentation et vidéo de secours.

10.2 Critères de validation de la V1

Domaine	Critère de validation
Intelligence artificielle	Une baseline et un modèle séquentiel sont comparés sur un jeu de test séparé, avec MAE, RMSE et R^2.
Détection d'anomalies	Un score d'anomalie est calculé à chaque cycle et évalué (précision, rappel, F1).
Temps réel	La simulation d'un moteur met à jour l'interface automatiquement, cycle après cycle, en moins de 2 secondes.
Health Score et alertes	Le passage sain → avertissement → critique est visible en direct et génère les alertes correspondantes.
Historique et traçabilité	Chaque prédiction est enregistrée avec son cycle et la version du modèle ; l'historique est consultable.
API	Toutes les routes de la V1 sont fonctionnelles et documentées.
Tests	Les fonctions critiques et le chemin complet de la simulation sont couverts par des tests.

Domaine	Critère de validation
Reproductibilité	L'application se lance sur une machine propre à partir du README et de Docker Compose.
Rigueur scientifique	Aucune métrique n'est inventée ; les limites du système sont clairement présentées.

10.3 Scénario de démonstration pour l'examen

Durée	Étape de la démonstration
~ 1 min	Présentation du problème : pannes imprévues et difficulté d'interprétation des données de capteurs.
~ 1 min	Présentation de l'architecture : données → modèles → API FastAPI → PostgreSQL → interface Angular.
~ 2 min	Lancement d'une simulation : évolution en direct des courbes, de la RUL et du Health Score.
~ 1 min	Déclenchement automatique d'une alerte critique et consultation de sa justification et de l'historique.
~ 1 min 30	Présentation des résultats réels : comparaison baseline / LSTM-GRU et évaluation de la détection d'anomalies.
~ 1 min	Conclusion : limites, reproductibilité et évolutions prévues (RAG, Digital Twin).

10.4 Livrables des versions suivantes

Version	Livrables	Critère de validation principal
V1.5	Corpus documentaire et index Qdrant, service RAG, pages Assistant IA et Documents, authentification, suivi MLflow, benchmark de questions.	Réponses documentaires toujours sourcées et cas d'abstention démontré.
V2	Vue Digital Twin 3D, module d'explicabilité, recherche hybride, extensions à d'autres jeux de données.	Fonctionnalités intégrées sans dégrader la stabilité de la V1.

11. Conclusion

Ce cahier des charges définit le cadre de réalisation de **SentinelX AI**, une plateforme de maintenance prédictive qui associe apprentissage automatique, suivi en temps réel et, à terme, assistance documentaire par IA générative. Le projet couvre l'ensemble de la chaîne d'un système d'intelligence artificielle : de la préparation des données NASA C-MAPSS à l'entraînement et à l'évaluation des modèles, jusqu'à leur intégration dans une application web construite avec Angular, FastAPI et PostgreSQL.

L'organisation en versions permet de concentrer l'effort sur un objectif clair et réaliste : livrer pour l'examen une **V1 complète, stable et évaluée**, intégrant la prédiction de la RUL, la détection d'anomalies, le Health Score, les alertes et le suivi des machines en temps réel. Le Maintenance Copilot RAG et le Digital Twin, qui complètent la vision du projet, sont planifiés comme des évolutions naturelles, que l'architecture retenue permettra d'intégrer progressivement.

Ce document servira de référence tout au long du projet et pourra être mis à jour en accord avec l'encadrant si des ajustements s'avèrent nécessaires.

Références

Datasets

- NASA Prognostics Center of Excellence — Data Set Repository : nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository
- NASA — CMAPSS Jet Engine Simulated Data : data.nasa.gov/dataset/cmapss-jet-engine-simulated-data
- UCI Machine Learning Repository — AI4I 2020 Predictive Maintenance Dataset : archive.ics.uci.edu/dataset/601/ai4i
- Zenodo — MIMII Dataset : zenodo.org/records/3384388

Technologies

- Angular — documentation officielle : angular.dev
- FastAPI — documentation officielle : fastapi.tiangolo.com
- PostgreSQL — documentation officielle : postgresql.org/docs
- scikit-learn : scikit-learn.org · PyTorch : pytorch.org
- Qdrant — recherche hybride et reclassement : qdrant.tech/documentation
- MLflow Tracking : mlflow.org/docs
- Docker Compose : docs.docker.com/compose

Support académique

- M. Selem, *Projet d'Intégration en Intelligence Artificielle*, support de cours — démarche : problème réel → données → exploration → modélisation → évaluation → intégration.